

Wie viel Wahrnehmung braucht ein NEAT-Agent?

Einfluss von Zielrichtung und Sichtradius auf das Lösen von Labyrinthen

Tobias Bandt
HAW Hamburg
Hamburg, Deutschland
eawp@tobiasbandt.de

Abstract

Ein mit NEAT evolvierter Agent löst ein 25×25-Labyrinth nur unzuverlässig. Diese Arbeit prüft, ob ihm dafür Wahrnehmung fehlt. Fünf Varianten mit Sichtradius 1 bis 3 und optionaler Zielrichtung treten auf denselben Karten gegeneinander an, ergänzt um eine Ablationsvariante und eine Studie zu vier NEAT-Parametern, zusammen 237 Läufe. Keine der Änderungen verschiebt messbar, welche Labyrinth gelöst werden. Die Zielrichtung bringt über zwanzig Karten 1,5 Karten ($p = 0,357$), ein größerer Sichtradius ändert die Lösungsmenge überhaupt nicht und kostet bis zum 3,8-Fachen an Rechenzeit. Ein auf den ersten zehn Karten überzeugender Effekt verschwand auf frischen Karten wieder. Was die Ergebnisse dominiert, ist die Schwierigkeit der einzelnen Karte.

Keywords

Neuroevolution, NEAT, Labyrinth, Wahrnehmungsradius, Evolutionäre Algorithmen

1 Einführung

Im Sommersemester 2026 wurde an der Hochschule für Angewandte Wissenschaften in Hamburg der Wahlpflichtkurs Evolutionäre Algorithmen von Prof. Dr.-Ing. Christian Lins angeboten. Im Verlauf dieses Kurses wurden Themen wie Hill Climbing, Differential Evolution, Genetische Algorithmen und NEAT in praktischen Übungen vertieft. Zum Kurs gehört ein Praktikum aus sechs aufeinander aufbauenden Aufgabenblättern. Diese Ausarbeitung knüpft an das dritte an, in dem mit NEAT [6] ein neuronales Netz evolviert wird, das einen Agenten durch ein zufällig erzeugtes Labyrinth steuert.

Bei der Bearbeitung ist mir aufgefallen, dass der Agent zwar lernt, nicht gegen Wände zu laufen, das Ziel aber nur selten und eher zufällig erreicht. Der Grund liegt in den Eingaben: Die acht Umgebungsfelder sagen dem Netz, wohin es sich bewegen *kann*, aber nicht, wohin es sich bewegen *sollte*. Ein vorwärtsgerichtetes Netz bildet dieselbe Eingabe immer auf dieselbe Ausgabe ab, also führen zwei lokal gleich aussehende Stellen im Labyrinth zwangsläufig zur selben Bewegungsrichtung – unabhängig davon, wo das Ziel liegt. Das Aufgabenblatt [3] nennt die acht Felder allerdings nur als Untergrenze und stellt ausdrücklich frei, dem Agenten „vielleicht etwas mehr“ zu zeigen.

Daraus ergibt sich die Fragestellung: Wie viel Wahrnehmung – in Form von Zielrichtung und Sichtradius – braucht der Agent, um ein Labyrinth zuverlässig zu lösen, und ab wann schadet mehr Eingabe mehr, als sie nützt? Der zweite Teil ist dabei genauso wichtig wie der erste, denn jede zusätzliche Eingabe vergrößert das Netz und damit den Suchraum, den NEAT durchsuchen muss. Meine Vermutung ist deshalb ein „Sweet Spot“. Mehr Wahrnehmung hilft bis zu einem Punkt und wird danach zum Nachteil. Ergänzend werden

einige NEAT-Parameter variiert. Kapitel 2 beschreibt die Aufgabe und den Stand des Codes, Kapitel 3 die Varianten und die vorab festgelegten Hypothesen, die Kapitel 4 bis 7 Umsetzung, Messwerte und Bewertung.

2 Ausgangszustand

2.1 Aufgabenstellung und Umsetzung

Das Aufgabenblatt verlangt, mit `neat-python` [4] Individuen mit mindestens acht Eingängen (die acht Umgebungsfelder) und vier Ausgängen (die vier Richtungen) zu evolvierten, eine Fitness-Funktion zu schreiben, die den Agenten schrittweise durch eine zufällig erzeugte Karte steuert, und Pfad sowie bestes Netz zu visualisieren. Der Agent hat ein festes Schrittbudget und bekommt die beste Bewertung, wenn er das Ziel erreicht. Andernfalls muss die Fitness sinnvoll skaliert werden, etwa über den Abstand zum Ziel oder die Anzahl erfolgreicher Schritte. Kartengröße, Start- und Zielpunkt sind frei wählbar, und das Blatt fordert explizit zum Experimentieren mit Zielfunktionen und Kartenparametern auf.

Meine Umsetzung baut auf der mitgelieferten Vorlage `agent-neat-vorlage.py` auf. Die Klasse `MapGenerator` erzeugt eine 25×25-Karte, in der jedes Feld mit 25 % Wahrscheinlichkeit eine Wand ist. Der Agent startet unten rechts und soll das Ziel oben links erreichen, gezeichnet wird über ein `Tkinter-Canvas`. Zwei Ungenauigkeiten der Vorlage seien erwähnt, weil sie beim Lesen des Codes irritieren: Die Marker 'S' und 'E' sind gegenüber der Agentensicht vertauscht, und die Richtungsnamen in `move()` passen nicht zur Zeichenrichtung des `Canvas`. Beides ist rein kosmetisch, da das Netz seine Ausgänge nur über deren Wirkung lernt. Als Einstieg in `neat-python` dient die im Aufgabenblatt genannte Kursliteratur, Kapitel 2 ab Seite 39 in [5].

`move()` prüft, ob das Zielfeld innerhalb der Karte und keine Wand ist. Ungültige Schritte lassen den Agenten stehen, verbrauchen aber trotzdem einen Schritt. Zusätzlich habe ich einen Schalter für Backtracking eingebaut. Aktiv ist die Variante ohne: Besuchte Felder gelten als gesperrt, damit ein Netz nicht einfach zwischen zwei Feldern pendelt. Im Gegenzug ist eine Sackgasse endgültig, denn ab dort ist jeder weitere Schritt ungültig.

2.2 Wahrnehmung und Fitness

Die Wahrnehmung steckt vollständig in `_get_map_env()` (Listing 1). Die Methode liest die acht Nachbarfelder aus, wertet Felder außerhalb der Karte als Wand und gibt einen Vektor der Länge acht zurück, passend zu `num_inputs = 8`.

```
1 def _get_map_env(self):  
2     env = []  
3     def get_value(x, y):  
4         if x < 0 or y < 0 or x >= len(self.map) \  
5             or y >= len(self.map[0]):
```

Tabelle 1: Wichtigste Parameter aus neat-config im Ausgangszustand.

pop_size	1000	num_inputs / num_outputs	8 / 4
fitness_threshold	40,0	num_hidden	2
feed_forward	False	activation	relu
conn_add_prob	0,5	conn_delete_prob	0,2
node_add_prob	0,3	node_delete_prob	0,1
compatibility_threshold	2,8	weight_mutate_rate	0,8
		elitism	2

```

6         return 1 # ausserhalb der Karte = Wand
7     elif self.map[x][y] in ('E', 'S'):
8         return 0
9     return self.map[x][y]
10 # alle acht Nachbarfelder, hier verkuerzt
11 for dx, dy in [(-1,1),(0,1),(1,1),(-1,0),
12               (1,0),(-1,-1),(0,-1),(1,-1)]:
13     env.append(get_value(self.pos_x+dx, self.pos_y+dy))
14 return env

```

Listing 1: Wahrnehmung im Ausgangszustand: acht Nachbarfelder, keine Zielinformation.

Diese Methode ist die zentrale Stellschraube dieser Arbeit: Sowohl ein größerer Sichtradius als auch eine Zielrichtung lassen sich hier einbauen, ohne den Rest des Programms zu ändern – lediglich `num_inputs` muss zur neuen Vektorlänge passen. Genauso wichtig ist, was nicht darin steht: keine Position, kein Abstand, keine Richtung zum Ziel.

Die Methode `run()` führt pro Agent bis zu `MAX_STEPS = 156` Schritte aus (bei 100 Schritten bliebe kaum Spielraum, da der kürzeste Weg schon 48 Schritte lang ist) und berechnet dabei die Fitness:

$$F = (d_{\text{start}} - d_{\text{best}}) + 10 \cdot [\text{Ziel erreicht}] + \sum_t (0,1 \cdot [\text{Schritt gültig}] - 0,1) \quad (1)$$

Der erste Term belohnt nicht den Abstand am Ende, sondern die größte je erreichte Annäherung, sodass ein Agent seinen Fortschritt nicht verliert, wenn er später in eine Sackgasse läuft. Der Bonus von 10 hebt erfolgreiche Agenten deutlich ab, und die Schrittbilanz bestraft effektiv nur ungültige Schritte. Bei einem Startabstand von $\sqrt{24^2 + 24^2} \approx 33,94$ läge die maximal erreichbare Fitness bei rund 43,94, und die konfigurierte `fitness_threshold` von 40,0 wäre ein brauchbares Abbruchkriterium für „gelöst“ – vorausgesetzt, der Agent kann das Zielfeld überhaupt betreten. Genau das ist im Ausgangszustand nicht der Fall, wie der folgende Abschnitt zeigt.

2.3 NEAT-Konfiguration und Schwachstellen

`eval_genomes()` baut aus jedem Genom ein Netz, lässt einen frischen Agenten laufen und schreibt dessen Fitness zurück. Alle Genome werden auf derselben Karte bewertet, da der Aufruf zum Neuerzeugen pro Generation auskommentiert ist. Das macht die Läufe vergleichbar, erlaubt einem Netz aber prinzipiell, die Karte auswendig zu lernen. Tabelle 1 zeigt die wichtigsten Parameter aus `neat-config`. Gerechnet wird mit 1000 Individuen über maximal 100 Generationen, mit vorzeitigem Abbruch beim Überschreiten der Fitness-Schwelle.

Damit ergeben sich vier Schwachstellen. Erstens fehlt jede Zielinformation, sodass lokal identische Situationen zwangsläufig zur

selben Entscheidung führen. Zweitens erkennt der Agent bei einem Sichtradius von einem Feld eine Sackgasse erst, wenn er schon darin steht. Drittens verfällt ohne Backtracking nach dem Festfahren das restliche Schrittbudget. Viertens steht `feed_forward` auf `False`, erlaubt also rekurrente Verbindungen, während der Code die Netze mit `FeedForwardNetwork.create()` aufbaut – solche Verbindungen gehen beim Netzaufbau verloren, ein Teil der Mutationen läuft also ins Leere.

Fünftens: *Das Zielfeld lässt sich gar nicht betreten.* Der Generator schreibt den Marker 'S' auf das Feld (0,0), das aus Sicht des Agenten das Ziel ist. Die Wahrnehmung behandelt diesen Marker ausdrücklich als begehrbar, `valid_move()` verlangt dagegen den Wert 0 – und in Python ist `'S' == 0` falsch. Wahrnehmung und Bewegung widersprechen sich also: Der Agent sieht ein freies Feld, auf das er nicht treten kann. In der Konsequenz kann der Bonus von 10 nie vergeben werden, die maximal erreichbare Fitness sinkt auf rund 32,9, und die Schwelle von 40,0 ist unerreichbar. Ein Kontrolllauf über 100 Generationen endet entsprechend mit einer besten Fitness von 22,24, wobei der Agent auf dem Nachbarfeld des Ziels stehen bleibt.

Die ersten beiden Punkte sind die Stellschrauben dieser Arbeit. Der dritte wird für alle Varianten gleich gehalten. Der vierte und der fünfte sind Fehler und werden vor der ersten Messung behoben (Kapitel 3).

3 Vorgehen

3.1 Varianten

Das Vorgehen folgt einem klassischen Versuchsaufbau: eine Referenz und darauf aufbauend Varianten, bei denen jeweils genau eine Stellschraube verändert wird. Die Hypothesen habe ich bewusst vor der ersten Messung notiert, weil sich Messergebnisse im Nachhinein fast immer irgendwie erklären lassen. Drei Dinge werden vorab geklärt, die selbst nicht untersucht werden. Erstens wird die Erreichbarkeit des Ziels hergestellt. Dazu entscheidet nicht mehr jede Stelle für sich, was begehrbar ist, sondern eine gemeinsame Methode, die Wahrnehmung und Bewegung benutzen – zwei getrennte Prüfungen derselben Eigenschaft können sonst erneut auseinanderlaufen. Zweitens wird `feed_forward` auf `True` gesetzt, passend zum verwendeten Netztyp. Ohne diese Korrektur wären besonders die Aussagen zu den Mutationsraten nicht interpretierbar. Drittens bleibt Backtracking für alle Varianten ausgeschaltet, wird also konstant gehalten statt korrigiert. Alles drei gilt auch für die Referenz V0, damit die Vergleichbarkeit erhalten bleibt.

Der Sichtradius r bezeichnet die Anzahl der Felder, die der Agent in jede Richtung sieht, was $(2r + 1)^2 - 1$ Umgebungsfeldern entspricht: 8 Felder bei $r = 1$, 24 bei $r = 2$ und 48 bei $r = 3$. Die Zielrichtung kommt als zwei auf $[-1, 1]$ normierte Eingaben Δx und Δy hinzu, jeweils die Differenz der Koordinaten geteilt durch `MAP_SIZE - 1`. Die Normierung sorgt dafür, dass die Zielrichtung dieselbe Größenordnung hat wie die Wandeingaben und nicht allein durch ihren Zahlenwert dominiert. Tabelle 2 zeigt die Varianten. V2 ist die Kontrolle zu V1: Sie vergrößert die Eingabe, ohne die Zielrichtung zu verraten, und zeigt damit, ob die *Art* oder nur die *Menge* der Information hilft.

Tabelle 2: Untersuchte Varianten. num_inputs muss zur Länge des Eingabevektors passen.

Variante	Beschreibung	r	Ziel	num_inputs
V0	Referenz (Ausgangszustand)	1	nein	8
V1	nur Zielrichtung	1	ja	10
V2	Radius 2 ohne Ziel	2	nein	24
V3	Radius 2 mit Ziel	2	ja	26
V4	Radius 3 mit Ziel	3	ja	50

3.2 Hypothesen

H1: Die Zielrichtung ist der entscheidende Faktor. V1 löst das Labyrinth deutlich zuverlässiger als V0 und braucht weniger Generationen, weil das Netz erst mit Δx und Δy zwischen lokal gleich aussehenden Situationen unterscheiden und eine Strategie der Form „gehe Richtung Ziel, weiche an Wänden aus“ überhaupt abbilden kann.

H2: Mehr Eingaben allein bringen wenig. V2 verbessert die Erfolgsquote gegenüber V0 nur geringfügig und bleibt hinter V1 zurück, obwohl V2 mehr Werte bekommt. Ein größerer Radius liefert mehr Details zur Wandstruktur, aber weiterhin keinen Hinweis auf die Zielrichtung.

H3: Der Nutzen zusätzlicher Wahrnehmung ist nicht monoton. V3 ist besser als V1, V4 dagegen wieder schlechter als V3. Radius 2 hilft, Sackgassen zu erkennen, bevor der Agent hineinläuft – ein direkter Vorteil, weil sie ohne Backtracking endgültig sind. Bei Radius 3 wächst der Eingabevektor auf 50 Werte, NEAT muss aus deutlich mehr möglichen Verbindungen die wenigen relevanten herausfiltern, und innerhalb desselben Generationenbudgets erwarte ich ein schlechteres Ergebnis. Das Maximum vermute ich bei $r = 2$.

H4: Die NEAT-Parameter wirken schwächer als die Wahrnehmung. Populationsgröße, `compatibility_threshold` sowie `conn_add_prob` und `node_add_prob` verschieben die Ergebnisse messbar, aber in kleinerem Umfang als der Wechsel von V0 auf V1, weil Parameter nur steuern, wie schnell und breit gesucht wird, und keine Strategie ermöglichen können, die sich mit den vorhandenen Eingaben nicht darstellen lässt.

3.3 Messgrößen und Versuchsaufbau

Die Parameterstudie zu H4 läuft nicht auf allen Varianten, sondern nur auf einer, und variiert jeweils einen Parameter um seinen Ausgangswert herum: `pop_size` 250/500/1000, `compatibility_threshold` 2,6/2,8/3,2, `conn_add_prob` 0,3/0,5/0,7 und `node_add_prob` 0,1/0,3/0,5. Eine vollständige Kombination aller Varianten mit allen Werten wäre bei 1000 Individuen und bis zu 100 Generationen zeitlich nicht sinnvoll durchführbar. Als Grundlage dient V1. Ursprünglich war dafür die beste Wahrnehmungsvariante vorgesehen, doch V1 und V3 erwiesen sich in der Hauptmessung als nicht unterscheidbar. Bei Gleichstand hätte die automatische Wahl V3 genommen und für denselben Erkenntnisgewinn rund 40 % längere Läufe gekostet.

Erfasst werden pro Lauf die *Erfolgsquote* (Anteil der Wiederholungen, in denen das Ziel innerhalb von 100 Generationen erreicht

wurde), die *Generationen bis zur Lösung*, die *beste Fitness* – als einzige Kennzahl auch für erfolglose Läufe aussagekräftig – und die *Schritte bis zum Ziel* des besten Agenten als Maß dafür, ob eine Lösung effizient oder eher zufällig zustande kam.

Jede Variante wird mit $N = 10$ Wiederholungen gerechnet, da NEAT durch Startpopulation, Mutation und Elternauswahl stochastisch ist und ein einzelner Lauf nichts belegt. Wiederholung i verwendet dabei Labyrinth i aus einer festen Liste von zehn vorab erzeugten Karten, zusammen mit dem zugehörigen protokollierten Seed für den NEAT-Lauf. Innerhalb eines Laufs bleibt die Karte unverändert, das Netz darf sie also weiterhin auswendig lernen. Zwischen den Wiederholungen wechselt sie.

Entscheidend ist, dass alle Varianten exakt dieselben zehn Paare aus Karte und Seed bekommen. Damit profitiert keine Variante von einem leichteren Labyrinth, und alle Läufe sind reproduzierbar. Eine einzige feste Karte für sämtliche Läufe hätte diese Vergleichbarkeit zwar auch hergestellt, aber das gesamte Ergebnis von der Schwierigkeit genau dieser Karte abhängig gemacht: Ist sie zu schwer, scheitern alle Varianten, ist sie zu leicht, lösen sie alle – in beiden Fällen misst der Versuch nichts. Über zehn Karten wird die Erfolgsquote dagegen zu einer Aussage darüber, ob der Agent Labyrinth löst, und nicht nur, ob er ein bestimmtes löst. Der Preis dafür ist, dass sich Kartenschwierigkeit und Zufall des Verfahrens in der Streuung vermischen und nicht mehr getrennt ausgewiesen werden können.

Ausgewertet wird je Variante über Mittelwert und Streuung der Messgrößen sowie über den Fitness-Verlauf pro Generation.

4 Durchführung

Der vollständige Quelltext liegt diesem Bericht als gepacktes Archiv bei, deshalb beschreibt dieses Kapitel die Umsetzung, ohne sie abzdrukken. Es handelt sich um eine Einzelarbeit, Entwurf, Implementierung, Messung und Auswertung stammen vollständig von mir.

Alle Varianten laufen über denselben Codepfad. Die Stellschrauben stehen als Konstanten am Kopf von `agent_neat.py`, nämlich `RADIUS`, `USE_GOAL_DIR`, `ALLOW_BACKTRACK` und `WALL_PROBABILITY`. Eine eigene Methode je Hypothese wäre naheliegender gewesen, hätte aber fünf Kopien derselben Logik erzeugt. Die driften im Lauf der Arbeit auseinander und entwerfen genau den Vergleich, um den es geht.

4.1 Änderungen am Code

Tabelle 3 fasst die Eingriffe zusammen.

Der wichtigste Nachweis ist der zur verallgemeinerten Wahrnehmung. Das Verhalten eines Agenten hängt ausschließlich vom Eingabevektor ab. Stimmt der neue Vektor an jeder Position mit dem alten überein, sind die Trainingsläufe zwangsläufig identisch, und das lässt sich prüfen, ohne einen einzigen vollen Lauf zu rechnen oder Seeds vergleichen zu müssen.

Ein Muster zieht sich durch mehrere Eingriffe: Statt dieselbe Eigenschaft an zwei Stellen zu prüfen, gibt es jeweils eine Quelle. `_is_free()` beantwortet für Wahrnehmung und Bewegung gemeinsam, ob ein Feld begehbar ist, und `input_size()` misst die Vektorklänge an einem Probe-Agenten, statt sie aus `RADIUS` nachzurechnen. Der ursprüngliche Fehler mit dem unbetretbaren Zielfeld

war genau ein Fall von zwei Prüfungen, die auseinandergefallen sind.

4.2 Karten, Seeds und Ablauf

Wiederholung i benutzt Labyrinth i und Lauf-Seed i . Beide Zufallsquellen sind getrennt: `MapGenerator.generate(seed)` zieht aus einem eigenen `random.Random(seed)` und fasst den globalen Strom nicht an, während `seed_run(seed)` den globalen Generator setzt, aus dem `neat-python` Startpopulation, Mutationen und Elternauswahl bezieht. Ursprünglich war eine feste Reihenfolge aus Neusetzen und Kartenbau geplant. Die Trennung ist robuster, weil der Runner beim Bauen von zehn Karten hintereinander keine Reihenfolge mehr verletzen kann, ohne dass es auffällt.

Vier Skripte teilen sich die Arbeit. `experiment.py` rechnet die Wahrnehmungsvarianten, `sweep.py` die Parameterstudie, `ablation.py` die Kontrollvariante V1K, und `auswertung.py` erzeugt aus den CSV-Dateien alle Abbildungen und Tabellen dieses Berichts. Im Batch bleibt die Tkinter-Oberfläche aus, weil `mainloop()` sonst blockiert. Jeder Lauf schreibt seine Zeile sofort, damit ein Abbruch nach Stunden nicht die ganze Messreihe kostet.

Zwei Prüfgrößen tauchen in den Ergebnissen mehrfach auf. Der *Vorzeichentest* zählt, auf wie vielen Karten die eine Variante gewinnt und auf wie vielen die andere. Sind beide gleich gut, ist jede Richtung gleich wahrscheinlich, und k gleichgerichtete Karten haben die Wahrscheinlichkeit $0,5^k$. Daraus folgt eine harte Grenze des Aufbaus: Unterscheiden sich zwei Varianten nur auf zwei Karten, ist Signifikanz von vornherein unmöglich. Der *Permutationstest* nach Fisher [1] mischt die Ergebnisse innerhalb jeder Karte neu und hält damit die Kartenschwierigkeit fest. Er beantwortet, wie oft allein durch Zufall so große Unterschiede zwischen den Konfigurationen entstehen wie die gemessenen.

4.3 Verworfenen Erweiterung

Die zehn Karten der Hauptmessung werden von vielen Konfigurationen gelöst, was die Varianten schlecht trennt. Naheliegender war, die Wanddichte anzuheben. Ein Pilot über acht Karten zeigte bei 0,30 und 0,35 aber das Gegenteil: Der Anteil der Karten, die überhaupt trennen, sank von 20 % auf 12 %, weil an die Stelle des Deckeneffekts ein Bodeneffekt trat. Die Sackgassen stiegen dabei von 30 auf 54 je Karte, während der kürzeste Weg mit 48 auf 53 Schritten fast gleich blieb.

Bei der Fitness sah 0,35 zunächst besser aus, V1 lag auf 7 von 8 Karten vorn. Für den entscheidenden Vorversuch habe ich das Kriterium deshalb vorher aufgeschrieben, nämlich mindestens 4 von 5 Karten je Vergleichspaar. Erreicht wurden 3 von 5. Damit blieb `WALL_PROBABILITY` bei 0,25, und der Wert ist seither eine Konstante statt eines beiläufig veränderlichen Parameters. Das vorab notierte Kriterium hat sich bewährt, weil das Ergebnis genau an der Grenze lag und ohne Festlegung in beide Richtungen deutbar gewesen wäre.

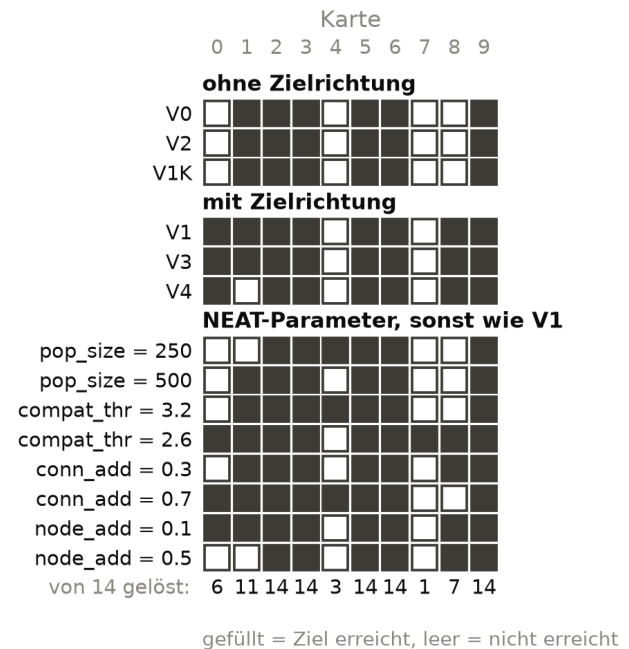


Abbildung 1: Alle 14 Konfigurationen gegen die zehn Karten der Hauptmessung, ein Quadrat je Lauf. Fünf Karten lösen alle Konfigurationen, zwei fast keine. Die Zeilen unterscheiden sich deutlich weniger als die Spalten. V0 und V2 lösen dieselben Karten, V1 und V3 ebenfalls.

5 Ergebnisse

In die Auswertung sind 237 Läufe mit zusammen 4,1 Stunden Rechenzeit eingegangen: 50 für die Hauptmessung, 30 für die Replikation auf frischen Karten, 20 für die Ablation, 80 für die Parameterstudie und 57 für die beiden Vorversuche zur Wanddichte. Dieses Kapitel nennt nur die Messwerte, die Deutung steht in Kapitel 6.

5.1 Wahrnehmungsvarianten

Tabelle 4 fasst die Hauptmessung zusammen. V1 und V3 lösen je 8 von 10 Karten, V0 und V2 je 6, V4 löst 7. Abbildung 1 zeigt dieselben Läufe einzeln statt gemittelt.

Der paarweise Vergleich fällt schärfer aus als die Quoten. V0 und V2 lösen exakt dieselbe Kartenmenge {1, 2, 3, 5, 6, 9}, V1 und V3 exakt dieselbe Menge {0, 1, 2, 3, 5, 6, 8, 9}. Der Sprung von Radius 1 auf Radius 2 verändert also in beiden Bedingungen nicht, welche Karten fallen. Er kostet dabei das 1,66-Fache an Rechenzeit (86 statt 52 s je Lauf ohne Zielrichtung, 79 statt 48 s mit).

V4 löst Karte 1 nicht, die alle anderen Varianten lösen. Auf den sieben Karten, die V3 und V4 gemeinsam lösen, braucht V4 im Mittel 52,7 statt 28,9 Generationen und ist auf sechs der sieben Karten langsamer. Ein Lauf von V4 dauert 181 s, das 3,8-Fache von V1.

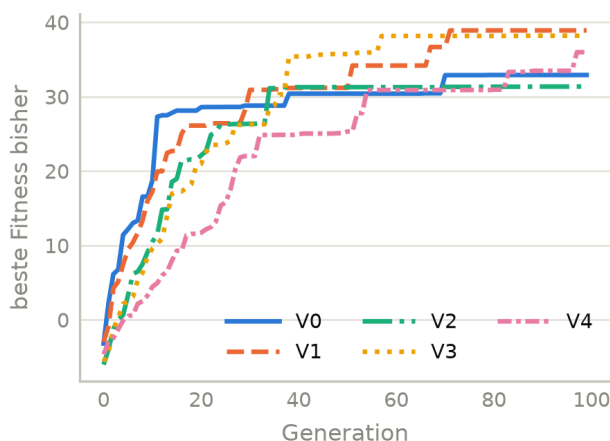
Abbildung 2 zeigt den Verlauf der bis dahin besten Fitness. V0 steigt in den ersten zehn Generationen am schnellsten, V4 durchgehend am langsamsten. V1 und V3 liegen am Ende am höchsten.

Tabelle 3: Eingriffe in den Code und die Messung, mit der ich sie abgenommen habe. Die ersten beiden beheben die Fehler aus Abschnitt 2.3, die übrigen schaffen die Stellschrauben.

Änderung	Nachweis
Zielfeld begehbar, <code>_is_free()</code> als einzige Quelle	vorher bei rund 32,9 gedeckelt, Kontrolllauf 22,24 und der Agent bleibt auf dem Nachbarfeld stehen, danach wird das Ziel erreicht
<code>feed_forward</code> auf <code>True</code>	30 Generationen, Population 150, gleicher Seed: vorher 62 % der Genome mit Zyklus, danach 0 %
Wahrnehmung über <code>RADIUS</code> verallgemeinert	3125 Positionen auf fünf Karten, Eingabevektor alt gegen neu bei Radius 1: 0 Abweichungen
<code>num_inputs</code> zur Laufzeit aus der gemessenen Länge	für Radius 1, 2 und 3 stimmen <code>num_inputs</code> , <code>input_keys</code> , gemessene Vektorlänge und ein echter Zwei-Generationen-Lauf überein
Zielrichtung als zwei optionale Eingänge	Vektorlänge 8, 24 und 48 jeweils plus 2, Werte durchgehend im Bereich $[-1, 0]$

Tabelle 4: Hauptmessung über zehn Karten. „Eing.“ ist die Länge des Eingabevektors, V1K die Ablationsvariante aus Abschnitt 5.4.

Variante	Eing.	gelöst	Gen. bis Ziel	beste Fitness	s/Lauf
V0	8	6/10	19.2 ± 25.1	32.91 ± 14.54	52
V1	10	8/10	33.5 ± 25.9	38.90 ± 10.64	48
V2	24	6/10	22.0 ± 9.9	31.36 ± 17.27	86
V3	26	8/10	30.0 ± 14.9	38.19 ± 12.32	79
V4	50	7/10	52.7 ± 28.3	35.98 ± 13.03	181
V1K	10	6/10	11.5 ± 7.5	33.78 ± 13.23	49

**Abbildung 2: Beste bisher erreichte Fitness je Generation, gemittelt über die zehn Läufe. Gelöste Läufe enden vorzeitig und werden mit ihrem Endwert fortgeschrieben, damit der Mittelwert nicht springt, wenn sich die Zahl der noch laufenden Wiederholungen ändert.**

5.2 Verhalten der Kennzahlen

Alle 139 gelösten Läufe erreichen exakt dieselbe Fitness von 43,94113, also genau den in Abschnitt 2.2 hergeleiteten Höchstwert $10 + \sqrt{24^2 + 24^2}$. Der beste ungelöste Lauf im gesamten Datensatz kommt auf 22,04. Zwischen 22,04 und 43,94 liegt kein einziger Messwert.

Damit fällt die konfigurierte Fitness-Schwelle von 40,0 mit dem Erreichen des Ziels zusammen: In allen 237 Läufen ist die Generation, in der die Schwelle überschritten wird, dieselbe wie die, in der zum ersten Mal ein Agent das Ziel betritt.

Die Schrittzahl des besten Agenten liegt zwischen 48 und 60, bei kürzesten Wegen von 48 bis 50, und stimmt in 108 der 139 gelösten Läufe mit dem kürzesten Weg überein. Die übrigen 31 Läufe brauchen bis zu zehn Schritte mehr und erreichen trotzdem dieselbe Fitness. Das entspricht der Formel: Der Abzug von 0,1 je Schritt und der Bonus von 0,1 je gültigem Schritt heben sich auf, sodass allein ungültige Schritte Fitness kosten.

Über die 14 Konfigurationen hinweg ist die mittlere beste Fitness fast vollständig durch die Zahl der gelösten Karten bestimmt ($r^2 = 0,95$). Innerhalb einer Lösungszahl streut sie um 0,3 bis 2,8 Punkte, zwischen benachbarten Lösungszahlen um 2,5 bis 5,5.

5.3 Unterschiede zwischen den Karten

Die zehn Karten der Hauptmessung tragen 22,6 % bis 29,0 % Wände, ihr kürzester Weg ist 48 oder 50 Schritte lang. Ihre Ergebnisse gehen weit auseinander (Fußzeile in Abbildung 1): Die Karten 2, 3, 5, 6 und 9 lösen alle 14 Konfigurationen, Karte 7 löst genau eine und Karte 4 drei. Nur die Karten 0, 1 und 8 werden von einem Teil der Konfigurationen gelöst und von einem anderen nicht.

Der Permutationstest über alle 14 Konfigurationen ergibt $p = 0,426$. Getrennt gerechnet liegt er für die sechs Wahrnehmungsvarianten bei $p = 0,46$ und für die Parameterstudie bei $p = 0,48$. Kein Paarvergleich einer einzelnen Konfiguration gegen V1 kommt im Vorzeichentest unter $p = 0,5$.

5.4 Replikation und Ablation

Auf den zehn frischen Karten 10 bis 19 kehrt sich das Bild der Hauptmessung um: V0 löst dort 8 Karten, V1 nur 6, und V1 liegt in der mittleren Fitness 6,55 Punkte unter V0. Über beide Kartensätze zusammen lösen V0 und V1 je 14 von 20 Karten bei einer Fitness-Differenz von $-0,28$. Auf zwei Karten gewinnt V1, auf zwei V0.

Die Ablationsvariante V1K entspricht V1 in Eingabelänge, Netzgröße und Startpopulation, bekommt an den beiden Zielrichtungseingängen aber den konstanten Wert $-0,5$ statt der echten Werte. Sie löst auf den Karten 0 bis 9 dieselben sechs Karten wie V0 und V2, über alle 20 Karten 13 von 20 gegenüber 14 von 20 bei V0. Auf den zwölf Karten, die V1 und V1K beide lösen, braucht V1K im Mittel

Tabelle 5: Parameterstudie auf den zehn Karten der Hauptmessung, jeweils ein Parameter gegenüber V1 verändert. „Eval.“ ist die Zahl der Fitness-Auswertungen bis zum Ziel, also pop_size mal Generationen. Sie ist nötig, weil Generationen über verschiedene Populationsgrößen hinweg nicht vergleichbar sind.

Parameter	Wert	gelöst	Gen.	Eval.	s/Lauf
Referenz	V1	8/10	33.5	34500	48
pop_size	250	6/10	25.7	6667	13
pop_size	500	6/10	13.2	7083	24
compatibility_threshold	2.6	9/10	45.6	46556	66
compatibility_threshold	3.2	7/10	31.4	32429	50
conn_add_prob	0.3	7/10	20.1	21143	46
conn_add_prob	0.7	8/10	16.2	17250	34
node_add_prob	0.1	8/10	32.8	33750	42
node_add_prob	0.5	6/10	17.7	18667	58

11,2 statt 16,1 Generationen und ist auf 7 der 12 Karten schneller ($p = 0,55$).

Abbildung 3 stellt die vier auf allen zwanzig Karten gemessenen Konfigurationen gegenüber. Auf den Karten 0 bis 9 lösen genau die Varianten mit Zielrichtung die Karten 0 und 8, auf den frischen Karten verschwindet diese Trennung. V3 erreicht dort dieselben acht Karten wie V0, und V1K löst Karte 11, die V1 verfehlt. Über alle zwanzig Karten lösen die beiden Varianten mit Zielrichtung im Mittel 15,0 Karten, die beiden ohne 13,5. Der Permutationstest dazu ergibt $p = 0,357$, und alle paarweisen Vorzeichentests liegen bei $p \geq 0,5$.

5.5 NEAT-Parameter

Tabelle 5 zeigt die Parameterstudie, Abbildung 4 ihre Erfolgsquoten. Der Referenzpunkt stammt aus der Hauptmessung und wurde nicht erneut gerechnet.

Keine der acht Konfigurationen unterscheidet sich im Vorzeichentest von der Referenz. Alle Werte liegen bei $p \geq 0,5$. Die höchste Quote erreicht `compatibility_threshold = 2,6` mit 9 von 10 Karten. Der Unterschied zur Referenz besteht aus genau einer Karte, nämlich Karte 7, an der die übrigen 13 Konfigurationen scheitern.

Deutlich sind dagegen die Aufwandsunterschiede. Die Generationen bis zum Ziel hängen im untersuchten Bereich nicht erkennbar von der Populationsgröße ab (25,7 bei 250, 13,2 bei 500, 33,5 bei 1000). Die Zahl der Fitness-Auswertungen bis zum Ziel folgt deshalb der Populationsgröße: 6 667 bei `pop_size = 250`, 7 083 bei 500 und 34 500 bei 1000. Ein Lauf dauert entsprechend 13, 24 und 48 Sekunden. Dabei ist zu beachten, dass die Obergrenze von 100 Generationen ein Generationenbudget ist: Ein Lauf mit 250 Individuen darf insgesamt 25 000 Agenten bewerten, einer mit 1000 Individuen 100 000.

6 Diskussion

6.1 Die vier Hypothesen

H1 ist widerlegt. Ich hatte die Zielrichtung für den entscheidenden Faktor gehalten, und auf den ersten zehn Karten sah es genau

danach aus. Die Karten 0 und 8 wurden von allen drei Konfigurationen mit Zielrichtung gelöst und von allen drei ohne verfehlt, eine saubere Trennung, zweimal. Auf zehn frischen Karten verschwindet sie. V3 löst dort dieselben Karten wie V0, und V1K holt eine Karte, die V1 verfehlt. Über alle zwanzig Karten bleiben 1,5 Karten Unterschied bei $p = 0,357$.

Der Grund liegt vermutlich darin, dass Start und Ziel fest sind. Weil das Ziel immer in derselben Ecke liegt, ist die *Richtung* dorthin konstant und das Vorzeichen beider Eingaben wechselt nie. Eine konstante Vorliebe für oben links kann das Netz gratis in seine Bias-Gewichte legen. Was Δx und Δy zusätzlich liefern, ist genau genommen die absolute Position. Die könnte helfen, zwei gleich aussehende Stellen auseinanderzuhalten, aber der Engpass ist ein anderer: Ohne Backtracking ist jede Sackgasse endgültig, und ob vor einem eine Sackgasse liegt, steht in den lokalen Wandfeldern und nicht in der Position. Lehman und Stanley beschreiben genau diese Klasse von Labyrinthaufgaben als täuschend: Eine Fitness, die den Abstand zum Ziel belohnt, führt die Suche in Sackgassen, weil der kürzeste Weg dorthin zunächst vom Ziel wegführt. Sie schlagen deshalb vor, statt des Ziels die Neuartigkeit des Verhaltens zu belohnen [2]. Die hier verwendete Fitness ist genau eine solche abstands-basierte. Bei einem wechselnden Ziel erwarte ich ein anderes Ergebnis, gemessen habe ich das nicht.

H2 ist gestützt, deutlicher als erwartet. Ein größerer Radius ändert nicht, *welche* Karten gelöst werden, sondern gar nichts: V0 und V2 lösen dieselbe Menge, V1 und V3 ebenfalls. Er kostet dabei 66 % mehr Rechenzeit. Von allen Ergebnissen ist das das sauberste, weil es nicht auf einem Mittelwert beruht, sondern auf identischen Mengen.

H3 ist zur Hälfte widerlegt. Der vermutete Sweet Spot bei Radius 2 existiert nicht, weil Radius 2 überhaupt nichts bringt. Die zweite Hälfte trifft zu: Radius 3 löst eine Karte weniger, braucht auf gemeinsamen Karten 52,7 statt 28,9 Generationen und dauert das 3,8-Fache. Der beste Wert ist damit der kleinste, nicht ein mittlerer.

H4 lässt sich so nicht prüfen. Sie behauptet, die Parameter wirkten *schwächer* als die Wahrnehmung. Nachdem die Wahrnehmung keinen messbaren Effekt hat, gibt es nichts, wogegen man vergleichen könnte. Was sich sagen lässt: Auch keine der acht Parameteränderungen verschiebt die Erfolgsquote nachweisbar. Verlässlich unterscheidbar ist allein der Aufwand, und dort ist das Ergebnis eindeutig. Die Generationen bis zum Ziel hängen nicht von der Populationsgröße ab, also verbraucht `pop_size = 1000` das Fünffache an Auswertungen für dasselbe Resultat.

6.2 Was der Versuch wirklich zeigt

Die eigentliche Aussage steckt nicht in den Varianten, sondern in den Karten. Zwölf von zwanzig Karten lösen alle Konfigurationen, vier keine, und nur vier trennen überhaupt. Die Schwierigkeit der Karte entscheidet, alles andere liegt darunter.

Daraus folgt eine Grenze, die ich vorher unterschätzt habe. Wenn nur 20 % der Karten trennen, braucht ein Vorzeichentest rund vier bis fünf gleichgerichtete Karten für Signifikanz. Bei zwanzig Karten und vier informativen ist das die Obergrenze dessen, was der Aufbau überhaupt zeigen kann. Ein Effekt, der kleiner ist als drei bis vier Karten Unterschied, wäre in diesen Daten unsichtbar geblieben, egal wie oft ich messe. Der Befund lautet also nicht „es gibt keinen

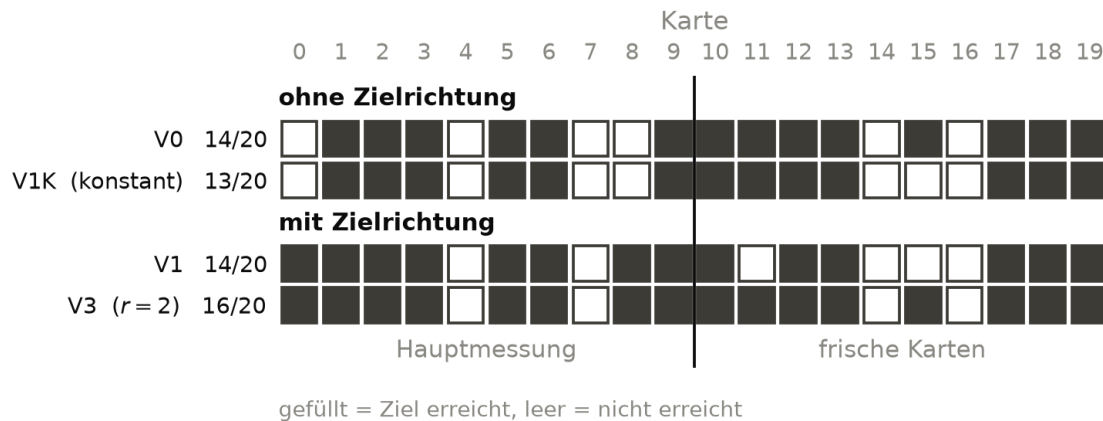


Abbildung 3: Die vier Konfigurationen, die auf allen zwanzig Karten gerechnet wurden. Links die zehn Karten der Hauptmessung, rechts zehn vorher ungenutzte. Die Trennung nach Zielrichtung auf den Karten 0 und 8 wiederholt sich rechts nicht.

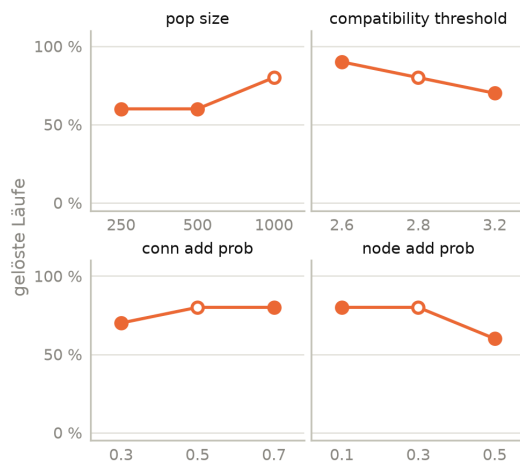


Abbildung 4: Erfolgsquote je Parameterwert. Der hohle Punkt ist der Referenzwert aus der Hauptmessung.

Effekt“, sondern „der vorhergesagte große Effekt existiert nicht, und ein kleiner wäre hier nicht nachweisbar“.

6.3 Grenzen und was ich anders machen würde

Drei Punkte schränken die Aussagekraft ein. Erstens hat die Fitness weniger Aussagekraft, als ich beim Entwurf annahm: Sie sättigt bei jedem gelösten Lauf am selben Wert, belohnt kurze Wege nicht, und ihre Schwelle fällt mit dem Zielerreichen zusammen. Über die Zahl der gelösten Karten hinaus trägt sie nur auf Karten bei, die niemand löst. Zweitens bleibt die Karte innerhalb eines Laufs konstant, das Netz darf sie also auswendig lernen. Der Versuch misst nicht, ob ein Agent Labyrinth löst, sondern ob er diese zwanzig löst. Drittens gibt es je Konfiguration und Karte nur einen Lauf, sodass sich der

Zufall des Verfahrens nicht direkt von der Wirkung der Konfiguration trennen lässt. Der Permutationstest umgeht das, indem er die Kartenschwierigkeit festhält, ersetzt aber keine Wiederholungen.

Rückblickend hätte ich die falsche Stellschraube untersucht. Die dominierende Größe ist nicht die Wahrnehmung, sondern das ausgeschaltete Backtracking, das Sackgassen tödlich macht. Es habe ich bewusst konstant gehalten. Für eine Fortsetzung wären ein wechselndes Ziel, ein Vergleich mit und ohne Backtracking und ein Budget in Auswertungen statt in Generationen die drei naheliegendsten Änderungen.

7 Fazit

Die Ausgangsfrage war, wie viel Wahrnehmung ein NEAT-Agent braucht und ab wann mehr Eingabe schadet. Die Antwort fällt anders aus als erwartet: In diesem Aufbau braucht er keine zusätzliche. Weder die Zielrichtung noch ein größerer Sichtradius verändern messbar, welche Labyrinth gelöst werden, und beide kosten Rechenzeit. Ein Sweet Spot existiert nicht, das Optimum ist die kleinste Wahrnehmung.

Fachlich am meisten mitgenommen habe ich aber etwas anderes. Zweimal sah ein Effekt in den Daten überzeugend aus, einmal mit einer rechnerischen Wahrscheinlichkeit von etwa 1 zu 400, und beide Male hat er sich auf frischen Karten in Luft aufgelöst. Aufgefallen ist das nur, weil das Kriterium jeweils vor dem Lauf feststand. Ohne diese Regel stünde in dieser Arbeit heute eine falsche Kernaussage.

Literatur

- [1] Ronald A. Fisher. 1935. *The Design of Experiments*. Oliver and Boyd, Edinburgh.
- [2] Joel Lehman and Kenneth O. Stanley. 2011. Abandoning Objectives: Evolution Through the Search for Novelty Alone. *Evolutionary Computation* 19, 2 (2011), 189–223.
- [3] Christian Lins. 2026. Evolutionäre Algorithmen, Aufgabenblatt 3. Praktikumsunterlagen, HAW Hamburg, Version 2.1.
- [4] Alan McIntyre, Matt Kallada, Cesar G. Miguel, Carolina Feher de Silva, and Marcio Lobo Netto. 2024. neat-python. <https://github.com/CodeReclaimers/neat-python>. Version 2.0.0.

- [5] Iaroslav Omelianenko. 2019. *Hands-On Neuroevolution with Python: Build high-performing artificial neural network architectures using neuroevolution-based algorithms*. Packt Publishing Ltd, Birmingham.
- [6] Kenneth O. Stanley and Risto Miikkulainen. 2002. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10, 2 (2002), 99–127.